50. Docker in Simple Words

1. What is Docker?

Docker runs software inside small, sealed boxes called containers. Each box holds the app plus everything it needs to run, like the right version, settings and libraries.

> **Real-life example: a lunch box.** Your mom packs food, spoon and sauce in one box. Wherever you take it (school, office, train), you open it and eat. Nothing is missing.
> Docker does the same for software: **pack once, run anywhere.**

The problem Docker solves

"*It works on my machine!*" 😕 Your friend has a different Node version, a different Postgres version, or Windows instead of Linux, so the project breaks on their machine.

With Docker, everyone runs the **same box**, so it works the same everywhere.

Where the name comes from

A **docker** is a dock worker who loads **shipping containers** onto ships. Before 1956, ships carried loose boxes, sacks and barrels, so loading was slow and things broke or went missing. Then the **standard steel shipping container** came in. Every ship, crane and truck could handle the same box, no matter what was inside.

Docker did the same for software: **one standard box** that any computer can run, whatever is inside (Node, Postgres, Redis, Python…).

2. Why was Docker introduced? (history)

Stage 1: Physical servers (before ~2000)

One app ran on one physical machine. If you wanted a new app, you bought a new server.

- 💸 Expensive. Most servers sat idle at 10–15% use.
- 🐢 It took weeks to buy and set up a machine.
- 💥 If two apps shared a server, they fought over versions and libraries.

Stage 2: Virtual Machines (2000s: VMware, VirtualBox, AWS EC2)

One big server was split into many **fake computers (VMs)**, each with its **own full operating system**.

- ✅ Better use of hardware, and apps were isolated from each other.
- ❌ Heavy. Each VM carries a whole OS (GBs of disk, lots of RAM).
- ❌ Slow to start (minutes).
- ❌ It still didn't fix "works on my machine". Dev, test and production VMs were set up by hand and slowly drifted apart.

Stage 3: Linux containers existed, but were hard to use

Linux already had the building blocks for lightweight isolation: `chroot` (1979), FreeBSD Jails (2000), Solaris Zones (2004), Google's **cgroups** (2006–07) and **LXC** (2008). Google ran everything in containers internally.

The trouble was that these tools were **complicated, low-level and only for experts**. There was no easy way to package an app and share it.

Stage 4: Docker arrives (2013) 🐳

Solomon Hykes and his team at **dotCloud**, a cloud hosting company, built Docker as an internal tool. They **open-sourced it in March 2013** (shown at PyCon). It spread very fast.

Docker didn't invent containers. It made them **easy**:

What Docker added	Why it mattered
Simple commands (`docker run`)	Anyone could use containers, not just Linux experts
Image format	A standard, shareable package of app + dependencies
Dockerfile	The setup is written as code, so it can be repeated and version-controlled
Docker Hub (2014)	Download Postgres, Redis or Node images in seconds, like npm
Layers + caching	Small downloads and fast rebuilds

What came after

- **2014:** Docker Compose (from a tool called Fig) for running many containers together. Google released **Kubernetes** to manage thousands of containers.
- **2015:** The **OCI** (Open Container Initiative) made the image format an open standard, so other tools (Podman, containerd) can run the same images.
- **Today:** Containers are the normal way to ship backend apps. AWS, Azure, GCP, Render and Railway all run Docker images.

```
Physical server  ⟶  Virtual Machine  ⟶  Container (Docker)
1 app = 1 machine     many full OSes       many apps, 1 shared OS
costly, slow          heavy, minutes       light, seconds
```

So, in one line: why Docker?

> **Docker was introduced to make shipping software as easy and reliable as shipping goods in a standard container:** build once, run the same on any laptop, server or cloud, start in seconds, and use far fewer resources than VMs.

3. How does Docker work inside? (simple)

A container is **not** a mini computer. It is just a **normal process** on your machine, with walls built around it by the Linux kernel:

Linux feature	What it does	Example
Namespaces	Decide what the process can **see** (its own files, network, process list)	Blinkers on a horse 🐴: it only sees its own lane
cgroups	Limit how much it can **use** (CPU, RAM)	A data cap on a SIM card 📱
Layered filesystem	The image is made of stacked read-only layers, with a thin writable layer on top	Transparent sheets stacked on each other

That's why containers are tiny and start in seconds: there is **no extra OS to boot**. They share your machine's kernel.

> **On Windows / Mac?** Containers need a Linux kernel. Docker Desktop quietly runs a small Linux VM (via WSL 2 on Windows), and your containers run inside it. You don't have to manage it.

The parts of Docker

Part	Job
Docker CLI	The `docker` command you type
Docker Engine (daemon)	The background service that actually builds and runs containers
Registry (Docker Hub)	Where images are stored and downloaded from

```
You ──► docker CLI ──► Docker Engine ──► pulls image from Docker Hub
                                     └──► starts container
```

4. Without Docker vs With Docker

Without Docker	With Docker
Download the Postgres installer, click Next, Next, Next…	`docker run postgres`: one command
Installs into your system for good	Stays inside a box. Delete the box and it's gone, clean.
Hard to run 2 versions together	Run Postgres 14 and 16 side by side easily
Teammate repeats the setup by hand	Teammate runs `docker compose up` and is done
"Works on my machine" problems	Same everywhere: laptop, server, cloud

5. The 4 words you must know

Word	Simple meaning	Real-life example
Image	A ready-made template / recipe (read-only)	A **recipe** or a cake **mould**
Container	A running copy made from an image	The actual **cake** made from the recipe
Docker Hub	An online store of images (postgres, redis, node…)	Like **npm** or the **Play Store**, but for images
Dockerfile	A file with steps to build *your own* image	Writing **your own recipe**

> One image can make **many** containers, just like one recipe can make many cakes.

```
Dockerfile  ──build──►  Image  ──run──►  Container (running app)
(recipe)                (mould)          (cake 🎂)

Docker Hub  ──pull──►  Image   (download a ready-made one)
```

6. Container vs Virtual Machine

	Container (Docker)	**Virtual Machine**
Size	Small (MBs)	Big (GBs)
Starts in	Seconds	Minutes
Has its own full OS?	No, it shares your OS kernel	Yes, a full OS
Example	A **room** in a flat (shares building, water, power)	A whole separate **house**

7. Install and check

On Windows, install **Docker Desktop** from docker.com and open it (the whale 🐳 icon should be running). Then check:

```
docker --version
docker run hello-world      # downloads a test image and runs it
```

8. Most-used commands

Images

```
docker pull postgres:16         # download an image
docker images                   # list downloaded images
docker rmi postgres:16          # delete an image
```

Containers

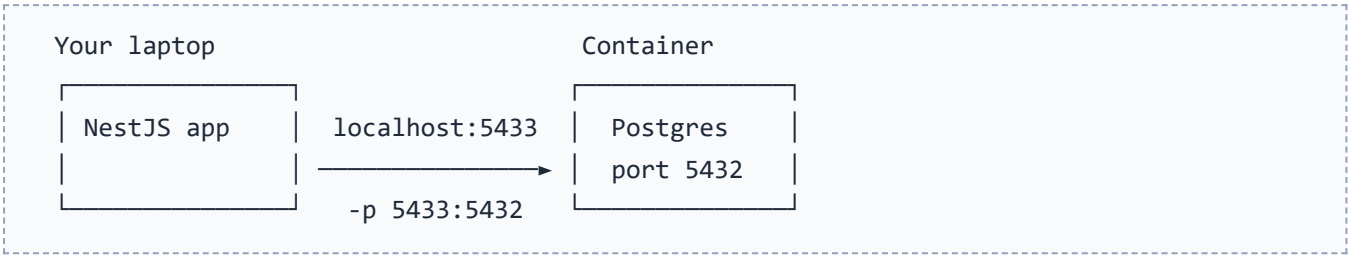
```
docker run -d --name redis -p 6379:6379 redis:7   # create + start
docker ps                       # running containers
docker ps -a                    # all containers (also stopped)
docker stop redis               # stop
docker start redis              # start again
docker rm redis                 # delete (must be stopped)
docker logs redis               # see output / errors
docker logs -f redis            # follow logs live
docker exec -it redis sh        # go inside the container
```

What do the `docker run` flags mean?

Flag	Meaning
`-d`	**Detached**: run in the background (your terminal stays free)
`--name redis`	Give the container a name (else Docker picks a random one)
`-p 6379:6379`	**Port**: `yourPC:container`. Opens a door from your laptop into the box.
`-e KEY=value`	**Environment variable**, e.g. a password
`-v name:/path`	**Volume**: save data outside the box so it isn't lost
`redis:7`	Image name : version (tag)

9. Ports in simple words

A container is a sealed box. Your NestJS app on your laptop can't reach inside unless you open a door.



`-p 5433:5432` means: **laptop port 5433 → container port 5432**. Your app connects to `localhost:5433`.

10. Volumes: don't lose your data!

> If you delete a container, **all the data inside it is deleted too**. That includes your database tables!

A **volume** is a folder that lives **outside** the box. Even if you delete the container, the data stays.

```
docker run -d --name pg \
  -e POSTGRES_PASSWORD=secret \
  -p 5432:5432 \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16
```

> **Example:** The container is a hotel room and the volume is your locker. You can check out of the room (delete the container) and your things stay safe in the locker.

11. Docker Compose: many containers, one file

Typing long `docker run` commands is painful. With **Docker Compose** you write everything once in `docker-compose.yml`.

```
services:
  postgres:
    image: postgres:16
    container_name: my-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7
    container_name: my-redis
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

Compose commands

```
docker compose up -d        # start everything
docker compose ps           # see status
docker compose logs -f      # watch logs
docker compose stop         # stop (keep containers)
docker compose down         # stop + remove containers (volumes stay)
docker compose down -v      # also delete volumes ⚠ data gone
```

> A new teammate just runs `git clone` and then `docker compose up -d`, and Postgres and Redis are ready. 🎉

12. Dockerfile: put your NestJS app in a box

So far we used **ready-made** images (postgres, redis). To package **your own app**, write a `Dockerfile` in the project root:

```
# 1. Start from an image that already has Node
FROM node:20-alpine

# 2. Work inside /app folder in the container
WORKDIR /app

# 3. Copy package files and install dependencies
COPY package*.json ./
RUN npm install

# 4. Copy the rest of the code and build
COPY . .
RUN npm run build

# 5. App listens on 3000
EXPOSE 3000

# 6. Command to start the app
CMD ["node", "dist/main.js"]
```

Also create `.dockerignore` (like .gitignore):

```
node_modules
dist
.env
.git
```

Build and run

```
docker build -t my-nest-app .            # build image from Dockerfile
docker run -d -p 3000:3000 --name api my-nest-app
# open http://localhost:3000
```

Dockerfile word	Meaning
`FROM`	Base image to start from
`WORKDIR`	Like `cd` into a folder inside the box
`COPY`	Copy files from laptop → box
`RUN`	Run a command while **building** the image
`EXPOSE`	Note which port the app uses
`CMD`	Command that runs when the **container starts**

> **Why copy `package.json` first?** Docker caches each step. If only your code changed, it skips `npm install`, so builds get much faster.

13. App + DB together in Compose

```
services:
  api:
    build: .                    # use our Dockerfile
    ports:
      - "3000:3000"
    environment:
      DB_HOST: postgres         # ⚠ service name, NOT localhost
      DB_PORT: 5432
      REDIS_URL: redis://redis:6379
    depends_on:
      - postgres
      - redis

  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7

volumes:
  pgdata:
```

> **Common confusion:** Inside Compose, containers talk to each other using the **service name** (`postgres`, `redis`), **not** `localhost`. `localhost` inside a container means "this same box".

Where your NestJS app runs	DB host to use
On your laptop (`npm run start:dev`)	`localhost`
Inside Docker Compose	`postgres` (the service name)

14. Common problems and fixes

Problem	Fix
`port is already allocated`	Something else uses that port. Change it to e.g. `-p 5433:5432`, or stop the other one.
`docker: command not found` / cannot connect	Open Docker Desktop and wait for it to start.
Container exits immediately	Run `docker logs <name>` to see the error.
Name already in use	`docker rm <name>`, then run again.
Data disappeared	You forgot a volume, or used `down -v`.
App can't reach DB in Compose	Use the service name, not `localhost`.

15. Quick summary

Question	Answer
What is Docker?	A tool to run apps in small sealed boxes (containers)
Who / when?	Solomon Hykes at dotCloud, open-sourced in March 2013
Why introduced?	VMs were heavy and Linux containers were hard to use. Docker made containers simple and shareable.
How it works	A normal process isolated by Linux namespaces + cgroups, with no extra OS
Benefits	Same setup everywhere, one-command install, clean removal
Image vs Container	Recipe vs the cake made from it
Port `-p A:B`	Laptop port A → container port B
Volume	Keeps data safe outside the container
Compose	One YAML file to run many containers
Dockerfile	Recipe to package your own app
Top 5 commands	`run`, `ps`, `logs`, `stop`, `compose up -d`